A Mini Project Report of

PCC-201-COM : Data Structures

Submitted by

Jadhav Shreyash Ashok (ROLL NO. 56)

Under the Guidance of

Prof. Tejal H. Patil



Department of Computer Engineering

BHARATI VIDYAPEETH'S
COLLEGE OF ENGINEERING, LAVALE, PUNE, 412115

Savitribai Phule Pune University

[2026-27]

BHARATI VIDYAPEETH
COLLEGE OF ENGINEERING, LAVALE, PUNE

CERTIFICATE

This is to certify that the **"Mini Project Report"** submitted by **Jadhav Shreyash Ashok (56)**, on **"Implementation of Snake and Ladder [BFS]"** is work done by him and submitted during **2026–2027** academic year, in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering** in Computer Engineering, at **Bharati Vidyapeeth's College of Engineering, Lavale, Pune.**

Prof. Tejal H. Patil
(Guide)

Dr. Lalit V. Patil
Head of Computer Engg. Department

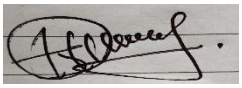
ACKNOWLEDGEMENT

It gives me immense pleasure to express my gratitude and thanks to all those who have helped in the timely completion of this Mini-Project work. I have great pleasure in expressing my deep sense of gratitude and indebtedness to **Prof. Tejal H. Patil**, my Guide, for her invaluable guidance, constant supervision and sustainedencouragement throughout the tenure of this Mini Project work.

I would like to express my thanks to **Dr. Lalit V. Patil**, Head of Department, Department of Computer Engineering, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for his constant support and encouragement.

I would like to express my thanks to **Prof. Dr. R. N. Patil**, Principal, Bharati Vidyapeeth's College of Engineering, Lavale, Pune for permitting me to take up the project work. I am thankful to my parents and friends for their kind help and support throughout the course.

Jadhav Shreyash Ashok
Student Name



ABSTRACT

The Snake and Ladder game is implemented in this project using the Breadth-First Search (BFS) algorithm to determine the shortest route from the starting position to the destination. The game board is represented as a graph, where each cell represents a vertex and each possible dice roll represents an edge between positions.

The system considers the effect of snakes and ladders during movement and explores all valid positions systematically. BFS uses a queue to process positions level by level, a visited set to avoid repeated exploration, and a parent map to store previous positions and reconstruct the path.

The main objective of this project is to demonstrate the practical application of graph traversal and shortest-path algorithms using Python. The system calculates the minimum number of dice throws required to reach the final cell and displays the resulting path. This project helps develop an understanding of BFS, graph representation, queue operations, and efficient problem-solving techniques.

TABLE OF CONTENTS

SR.NO.	CONTENT	PAGE NO.
	Acknowledgement	i
	Abstract	ii
	Content	iii
	List Of Figures	iv
1	Introduction	1
	a) Overview	
	b) Problem Statement	
	c) Objective	
2	Literature Survey	2
3	UML Diagram	3
4	System Requirement	4
5	System Design	5
	a) Existing system	
	b) Proposed system	
	c) Working Diagram	
	d) System code	
6	Result	8
7	Application	9
8	Future Scope	10
9	Conclusion	11
10	References	12

LIST OF FIGURES

Figure	Description	Page No.
Fig 3.1	UML Class Diagram	3
Fig 5.1	Working Diagram of Snake and Ladder BFS	6
Fig 5.2	System Code	7
Fig 6.1	Result (Demo Program and Output)	8

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Literature Survey	2
6.1	Top Keywords Obtained from the Demo Text	8

1. INTRODUCTION

1.1 Overview

The project “Implementation of Snake and Ladder using BFS” focuses on finding the shortest path from the starting cell to the final cell of a Snake and Ladder board. The board is represented as a graph, where each cell represents a vertex and each possible dice roll represents an edge between positions. Snakes move the player backwards, while ladders move the player forwards. The Breadth-First Search (BFS) algorithm explores possible positions systematically to determine the minimum number of dice throws required to reach the destination.

1.2 Problem Statement

The main problem is to find the minimum number of dice throws required to reach the final square from the starting position on a Snake and Ladder board. Exploring every possible sequence of dice rolls can be time-consuming and may involve repeated calculations. To solve this problem efficiently, the project uses Breadth-First Search (BFS), which explores possible positions level by level and identifies the shortest path in terms of dice throws.

1.3 Objectives

The main objectives of this project are:

- To represent the Snake and Ladder board as a graph.
- To model possible dice rolls as transitions between board positions.
- To implement snake and ladder mappings to determine the player's actual destination after each move.
- To use a queue for systematic BFS traversal.
- To maintain a visited set to avoid processing the same position repeatedly.
- To use a parent map to reconstruct the shortest path from the starting cell to the destination.
- To calculate and display the minimum number of dice throws required to reach the final cell using Python.

2 LITERATURE SURVEY

Graph traversal algorithms are widely used to explore networks and find paths between connected states. Breadth-First Search is particularly suitable for unweighted graphs because it explores all vertices at one distance before moving to the next. This survey summarizes concepts relevant to modelling Snake and Ladder as a graph-search problem.

Table 2.1 summarizes the main references that form the theoretical background of this project.

Year	Paper Name	Author	Conclusion
2025	Graph-Based Modelling and Shortest-Route Analysis of the Snakes and Ladders Game	Yaowei Zhang	The research demonstrates how graph structures can represent game movements and how BFS can identify a route with the fewest dice throws. It also discusses the use of Markov chains to analyse possible game outcomes.
2025	Graph Search Methods and Their Role in Network Exploration	Chreesk Sabah M. Ali andIbrahim Mahmood Ibrahim	The study explains the working principles of BFS and DFS, highlighting their efficiency, computational requirements, strengths, and limitations in exploring graph structures.
2025	Performance Evaluation of BFS and DFS for Shortest-Path Problems	Renad AlKahtani et al.	The research examines the behaviour of BFS and DFS when searching for paths in unweighted graphs. It highlights that BFS guarantees a shortest path in terms of edges, whereas DFS does not always provide an optimal path.
2025	An Overview of Path-Planning Techniques for Navigation and Search	Kenneth Christopher Ugwoke, Nwojo Agwu Nnanna, and Saleh El-Yakub Abdullahi	The study discusses different path-planning approaches, including classical search algorithms such as BFS. It explains how these techniques help identify routes between starting and destination points in different problem environments.

3 UML DIAGRAM

The UML class diagram below represents the Snake and Ladder BFS solver, including the snake and ladder mappings, BFS search procedure, queue, visited set and parent map used to reconstruct the shortest path.

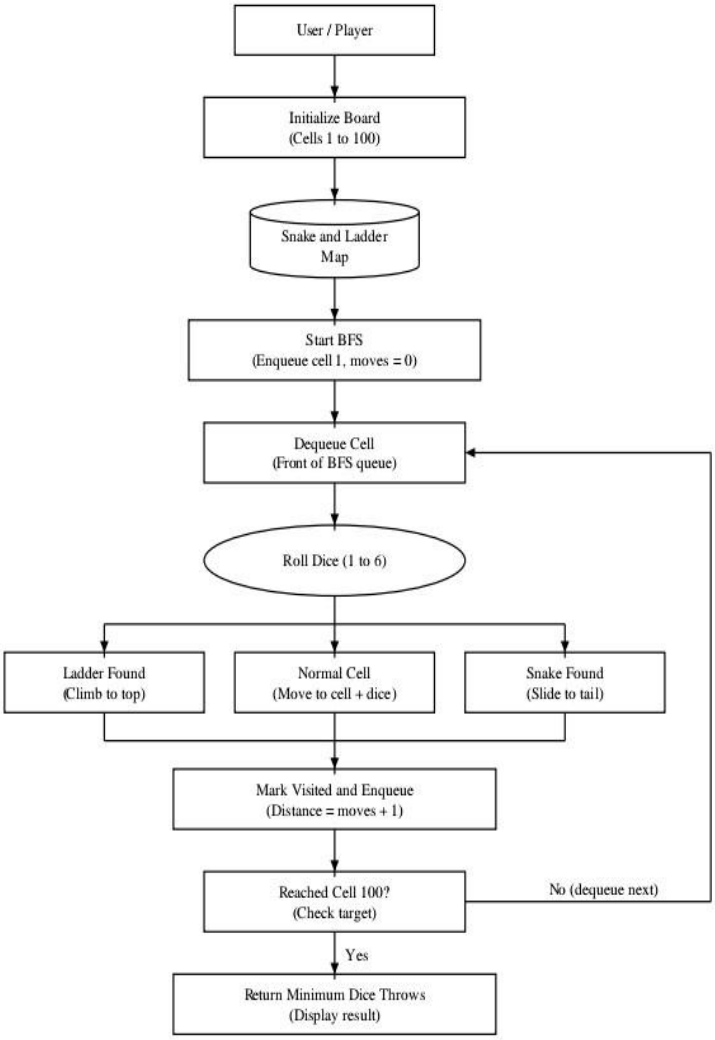


Fig 3.1: UML Class Diagram

4 SYSTEM REQUIREMENT

4.3 HARDWARE REQUIREMENTS

1. Operating System: Windows 11 64-bit
2. Processor Type: Intel Core
3. Recommended: Core 2 Duo or higher
4. Processor speed: 2.30 GHz or higher
5. RAM: Minimum 512 MB
6. Hard disk: 512 GB or more

4.4 SOFTWARE REQUIREMENTS

1. Jupyter Notebook (JupyterLab)
2. Language: Python 3 (ipykernel)
3. Python modules: re, collections

5 SYSTEM DESIGN

Existing System:

In a basic approach, the words of a text are stored in a list and, for every word, the list is searched to find whether the word already exists so that its count can be increased. This linear search needs O(n) time for every word, so counting a whole document takes $O(n^2)$ time in the worst case. Manual counting or spreadsheet formulas are also slow and error-prone for large texts, and they do not remove common words automatically.

Key approaches commonly used for solving the Snake and Ladder path problem:
Manual play (depends on repeated dice rolls)

- ✓ Brute-force sequence exploration (may repeat states)
- ✓ Breadth-First Search using a queue (explores states level by level)
- ✓ Visited set and parent map (avoid revisits and reconstruct the path)
- ✓ Library classes such as collections.Counter

Proposed System:

The proposed system models the board as a graph. Starting from cell 1, BFS examines each possible dice result from 1 to 6, applies snake or ladder transfers, and adds each unvisited destination to the queue. The parent map is used to reconstruct the shortest route once the final cell is reached.

- ✓ Initialize the queue with cell 1 and mark it visited.
- ✓ Generate valid dice-roll destinations for the current cell.
- ✓ Apply a snake or ladder transfer to each destination.

Time Complexity:

Operation	Time	Space
Tokenize text (n words)	O(n)	O(n)
Count with hash table	O(n) average	O(k)
Sort k unique words	O(k log k)	O(k)
top_n (count + sort + slice)	O(n + k log k)	O(k)

Working Diagram:

Fig 5.1 shows the working of the system. BFS starts at cell 1, generates possible dice moves, applies snake or ladder transfers, marks new cells as visited, and continues with a queue until the final cell is found. The parent map then reconstructs the shortest path.

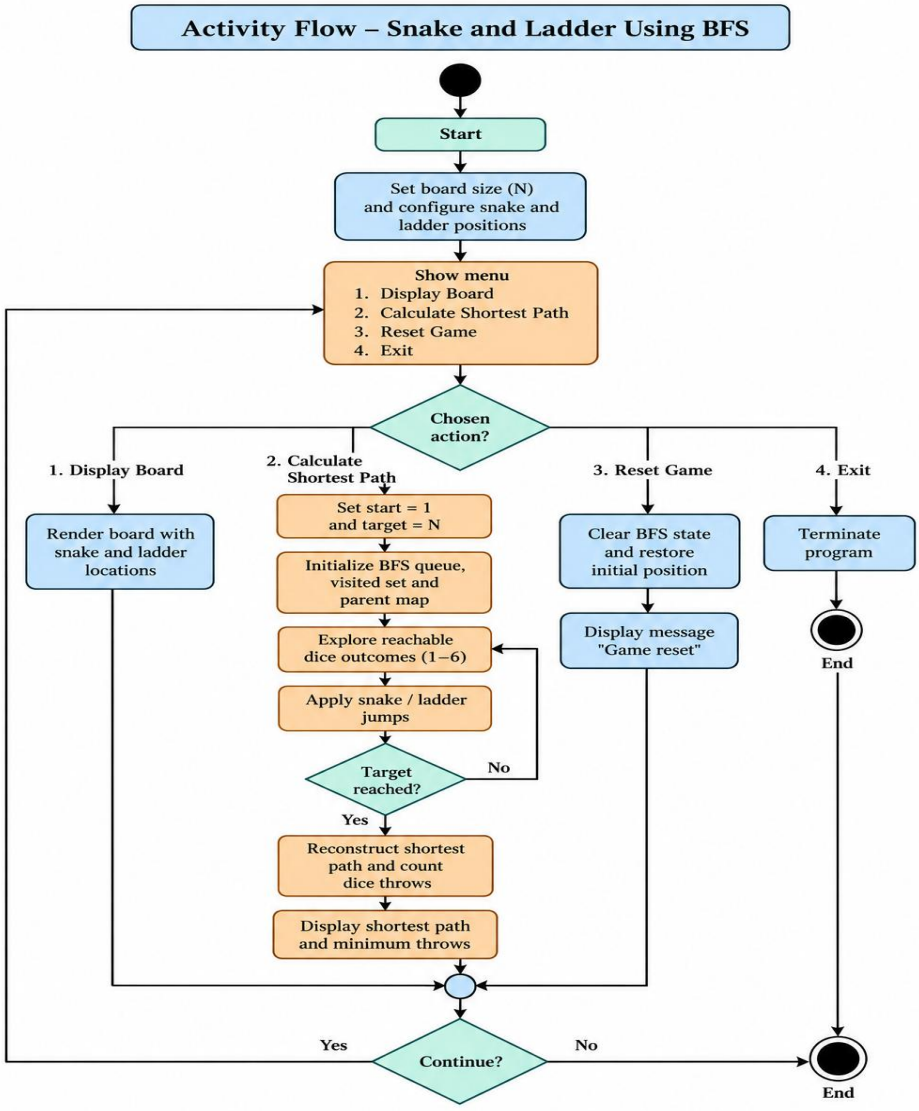
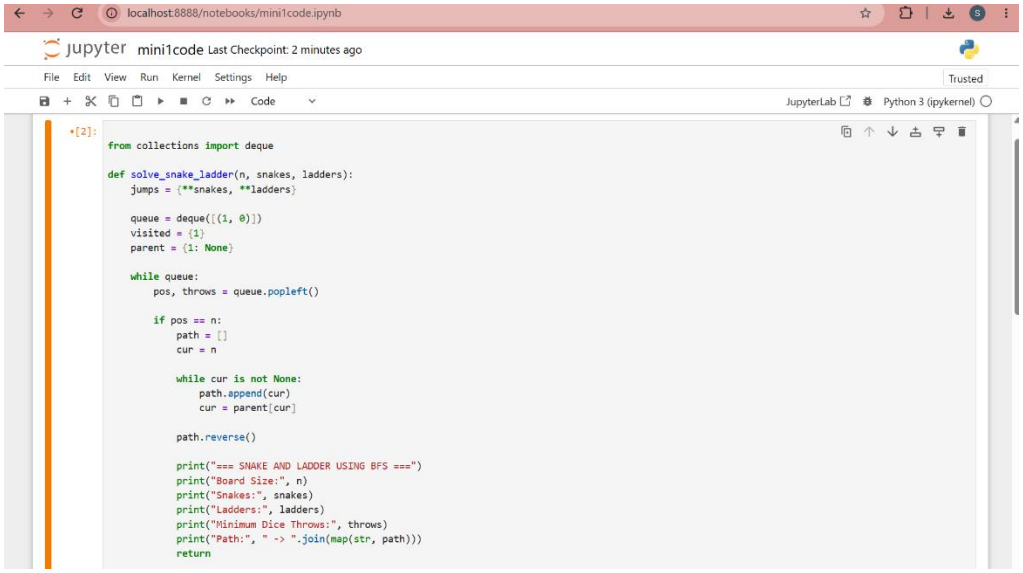


Fig 5.1: Working Diagram of Snake and Ladder BFS

SYSTEM CODE



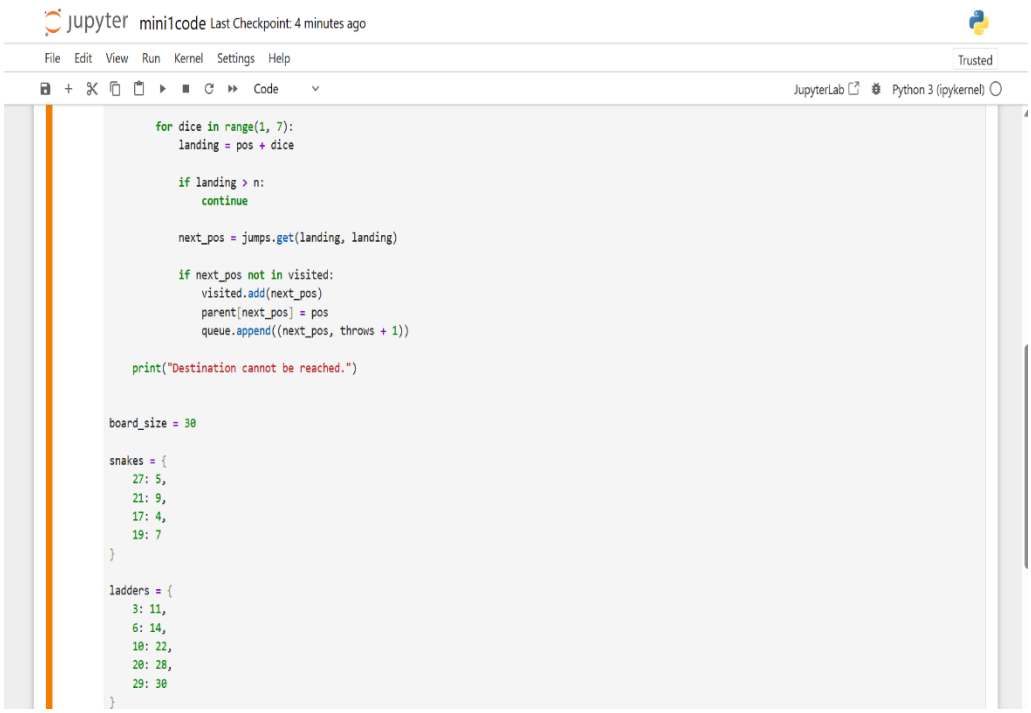


Fig 5.2(a): System code – snake/ladder mappings and BFS function

6 RESULT

The program explores reachable board positions using BFS and reports the minimum number of dice throws required to reach the final square. Each position is visited at most once, and the stored parent information is used to display the path found. The result depends on the snake and ladder positions defined for the board.

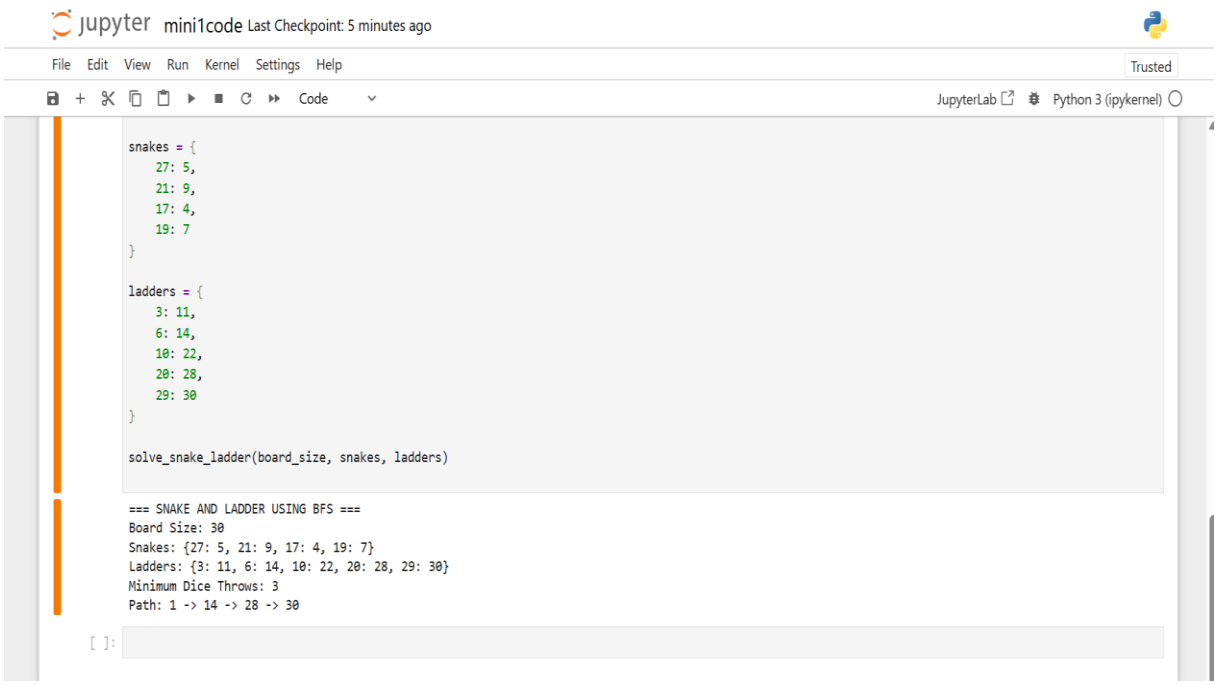


Fig 6.1: Result (Minimum Dice Throws and Shortest Path)

7 APPLICATIONS

Breadth-First Search (BFS) and graph modelling are useful for solving problems that involve exploring connected states and finding the shortest path. Their applications include:

- Shortest Path Finding: Finding the shortest route between two locations in unweighted graphs, maps, and networks.
- Puzzle and Board Games: Solving games such as Snake and Ladder by representing each board position as a graph vertex.
- Network Traversal: Exploring connected devices, websites, and nodes in computer networks.
- Maze Solving: Exploring possible paths through a maze to find a route from the starting point to the destination.
- Social Network Analysis: Finding connections between users and identifying relationships within a network.
- Game Development: Exploring possible game states and implementing basic decision-making strategies in game AI.

8 FUTURE WORK

The project can be extended in the following ways:

- Develop a graphical game board that visually displays snakes, ladders, player positions, and the shortest path identified by BFS.
- Allow users to customize the board size and define their own snake and ladder positions.
- Compare BFS with DFS and other pathfinding algorithms based on execution time, memory usage, and the number of explored positions.
- Create an interactive user interface that simulates dice rolls, player movements, and the step-by-step execution of BFS.
- Extend the system to support multiple players and display their positions and progress throughout the game.
- Add features to analyse different board configurations and evaluate how snakes and ladders affect the minimum number of dice throws.

9 CONCLUSION

In this project, the Snake and Ladder game was implemented using Breadth-First Search (BFS) by representing the board as a graph. The algorithm uses a queue to explore possible moves, a visited set to prevent repeated positions, and a parent map to reconstruct the shortest path.

The project demonstrates how BFS efficiently finds the minimum number of dice throws required to reach the destination. It also provides practical knowledge of graph representation, queue operations, visited states, and shortest-path algorithms.

10 REFERENCES

[1] Y. Zhang, "Research on Strategy Optimization for the Snakes and Ladders Game Based on Graph Theory and Markov Chains," *Transactions on Computer Science and Intelligent Systems Research*, vol. 11, pp. 403–409, 2025. DOI: https://doi.org/10.62051/5czp3d23

[2] C. S. M. Ali and I. M. Ibrahim, "A Review of Graph Traversal Algorithms: Techniques and Applications in Network Analysis," *Asian Journal of Research in Computer Science*, vol. 18, no. 3, pp. 61–72, 2025. DOI: https://doi.org/10.9734/ajrcos/2025/v18i3578

[3] R. AlKahtani, A. Alhabdan, M. Alosami, W. Alshammari, A. A. Abdo, and L. Hamdi, "Comparative Analysis between BFS and DFS-Shortest Path Algorithms," in *Proceedings of the 2025 IEEE Open Conference of Electrical, Electronic and Information Sciences (eStream)*, 2025. DOI: https://doi.org/10.1109/eStream66938.2025.11016860

[4] K. C. Ugwoke, N. A. Nnanna, and S. E.-Y. Abdullahi, "Simulation-Based Review of Classical, Heuristic, and Metaheuristic Path Planning Algorithms," *Scientific Reports*, vol. 15, article 12643, 2025. DOI: https://doi.org/10.1038/s41598-025-96614-2

[5] V. Nur'aini and F. Haryani, "The Creation of the SnaPhys (Snake Physics) Game as a Learning Medium for Physics on Renewable Energy Topics," *Journal of Advances in Physics Education*, vol. 1, no. 1, pp. 13–24, 2025. DOI: https://doi.org/10.23960/jape.v1i1.364